LABORATORIO 3.3

Sistema integral de observabilidad, AIOps, seguridad de red, SRE y chaos engineering

Informe ejecutivo

Este documento describe la solución implementada en este repositorio: tres microservicios Java, un pipeline OpenTelemetry local y la infraestructura preparada para Google Cloud y Amazon Web Services. Separa lo que el código y las pruebas unitarias ya demuestran de lo que quedó escrito y todavía no se ejecutó.

Dato	Valor en este documento
Objetivo del documento	Explicar la arquitectura, la instrumentación, las reglas de detección, los experimentos de caos y el grado de madurez, con el alcance real de la evidencia.
Fecha de elaboración	9 de octubre de 2026.
Programa e institución	No constan en el repositorio. No se inventan.
Autor	No consta un nombre personal en el repositorio. No se inventa.
Despliegue cloud	Preparado en Terraform y manifiestos. No aplicado.

Contenido

El informe sigue el orden en que se investigaría un incidente: primero el sistema, después las señales, luego la detección, la seguridad, el experimento y la madurez.

1. Resumen ejecutivo
2. Objetivos y alcance
3. Arquitectura observable
4. Instrumentación OpenTelemetry
5. Correlación entre métricas, logs y trazas
6. AIOps y detección de anomalías
7. Observabilidad de red y de seguridad
8. Chaos engineering, MTTD, SLO y error budget
9. Evaluación de madurez
10. Roadmap de 90 días
11. Conclusiones y recomendaciones
12. Referencias técnicas

Cómo leer las evidencias

Cada afirmación del informe cae en una de tres clases. Implementado y verificado: el código existe y una prueba de esta máquina lo ejecutó. Implementado y no ejecutado de punta a punta: el archivo está en el repositorio, pero el stack Docker no llegó a arrancar. Preparado para cloud: Terraform o un manifiesto describen el recurso y no hubo apply. Cuando una medición no existe, el texto dice cómo obtenerla y no rellena el número.

Clase	Ejemplos de este informe	Dónde comprobarlo
Verificado	Compilación y pruebas Maven; fórmula del error budget; compuertas del caos.	mvn test, el 9 de octubre de 2026.
No ejecutado de punta a punta	Traza en Tempo, alerta firing, MTTD, tableros con serie.	PASOS.md, secciones B y C, cuando el registro de imágenes responda.
Preparado, no desplegado	GKE, Cloud SQL, ECS, RDS, flow logs, Service Connect, Cloud Service Mesh.	terraform plan. No apply en el ensayo.

Tabla 0. Clasificación usada en todo el documento. Un archivo YAML no cambia de clase por sí solo.

1. Resumen ejecutivo

El problema es operar un flujo de pedidos distribuido sin poder decir, ante una degradación, qué servicio falló, si el fallo es de latencia o de errores, y si esa degradación consume el presupuesto de error. Una métrica aislada no basta: hace falta el mismo identificador de traza en el log y en el backend de trazas, y una alerta que no se dispare con el primer punto de una serie vacía.

La solución de este laboratorio es una cadena de tres procesos Spring Boot. service-a recibe GET /api/orders/{orderId}, service-b prepara el cumplimiento y puede inyectar una espera de 200 ms, y data-service es el único que abre JDBC contra MySQL. El Java Agent 2.31.1 instrumenta HTTP y el driver. El código añade tres spans de negocio. Un Collector 0.162.0 reparte trazas a Tempo, métricas a Prometheus y logs a Loki. Grafana aprovisiona dos tableros. Prometheus evalúa umbrales fijos y una regla que exige, a la vez, un error rate por encima de su baseline más dos desviaciones estándar y un p99 de service-a por encima de 150 ms.

El estado real, al 9 de octubre de 2026, es el siguiente. **mvn test** terminó con código 0. Cubre las compuertas del caos, la espera de 200 ms, el error inyectado antes de tocar la base y la lectura del pedido ord-001 sobre H2. El arranque con Docker no se completó: el registro respondió 504 al autorizar la descarga de maven:3.9.9-eclipse-temurin-21 y de eclipse-temurin:21-jre-jammy. Por eso este informe no trae una traza de Tempo, una captura de Grafana, un MTTD medido, un conteo de alertas firing ni un porcentaje de reducción de ruido. Tampoco hay un apply de Terraform.

Capacidad	Estado
Tres servicios y regla de stock	Implementada. Probada en unidad con H2, no contra MySQL en contenedor.
Agente, spans de negocio y logs JSON	Implementados en imagen y código. El pipeline completo no se observó en ejecución.
Regla baseline + 2σ y alertas estáticas	Escritas en Prometheus. Ninguna llegó a estado firing en esta sesión.
Enriquecimiento con trace_id	Script listo. No se ejecutó contra una alerta real.
Caos de 200 ms y de 10 %	Implementado y apagado por defecto. Las compuertas pasaron en pruebas unitarias. No hubo corrida contra Prometheus.
SLO 99 % y fórmula de presupuesto	La fórmula está probada con 1000 peticiones y 10 fallos. No hay ventana experimental medida.
GKE, Cloud SQL, ECS, RDS, flow logs	Versionados. No creados.
Service mesh L7	Descartado App Mesh por fin de soporte. Cloud Service Mesh y Service Connect documentados y no aplicados.

Tabla 1. Capacidades y estado de verificación al elaborar el informe.

2. Objetivos y alcance

2.1 Objetivo general

Construir un sistema de laboratorio en el que una solicitud de pedido pueda seguirse desde la entrada HTTP hasta la sentencia SQL, y en el que una degradación controlada pueda detectarse, explicarse y revertirse sin dejar el experimento encendido por accidente.

2.2 Objetivos específicos

- Instrumentar los tres servicios con el agente Java y con spans de negocio que no dupliquen el span JDBC.
- Propagar el contexto W3C entre service-a, service-b y data-service.
- Exportar métricas, logs y trazas por OTLP hacia un Collector con backends distintos.
- Definir una anomalía que exija dos condiciones a la vez: error rate estadístico y latencia p99 sobre el SLO.
- Separar la telemetría de aplicación, la de proxies y la de flujo de red, y no presentar como desplegado un mesh que no está aplicado.
- Dejar dos experimentos reversibles, apagados por defecto, y una fórmula explícita de error budget.
- Calificar ocho dominios de madurez solo con evidencia verificada.

2.3 Alcance funcional

El caso de negocio es consultar un pedido y decidir si el inventario alcanza. La semilla local carga ord-001 a ord-005. ord-001 corresponde al SKU LAPTOP-X1. La respuesta incluye si el stock es suficiente y, cuando hay un span válido, el traceId. La autenticación de laboratorio es un único endpoint POST /api/session: un token distinto del configurado responde 401 y cuenta el fallo. No es un proveedor de identidad.

2.4 Limitaciones declaradas

- El entorno local usa MySQL 8.4 en Docker. Ese motor no representa Cloud SQL ni Amazon RDS.
- Un proceso de data-service activa un solo perfil Spring: local, gcp o aws. No abre las dos bases cloud a la vez.
- No hay escáner de CVE conectado. El tablero de seguridad lo dice en un panel de texto y no muestra un número.
- Security Command Center y Security Hub no están activados. Sus hallazgos no se simulan.
- H2 en las pruebas de Maven no sustituye al MySQL del compose.
- La ausencia de credenciales y el fallo del registro de imágenes impiden afirmar un despliegue o un MTTD.

3. Arquitectura observable

service-a es el único punto de entrada del flujo de negocio. No tiene URL de base de datos. Llama a service-b en GET /api/fulfillment/{orderId}. service-b, si el experimento de latencia está armado y encendido, duerme 200 ms dentro del span fulfillment.prepare y después llama a data-service en GET /api/records/orders/{orderId}. data-service ejecuta el JOIN de orders e inventory y compara available con quantity. Si el experimento de error está activo y la petición trae la cabecera de laboratorio, el fallo se produce antes de la consulta.

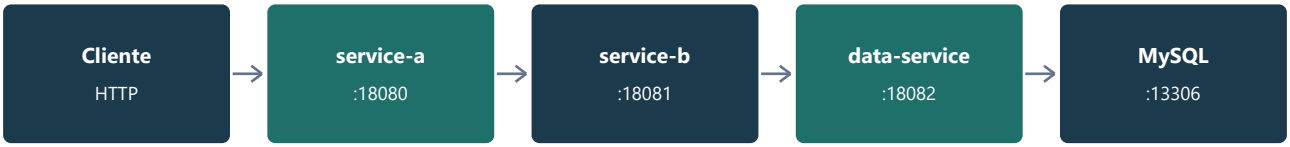


Figura 1. Recorrido de una consulta de pedido. Los puertos son los publicados en el host.

Servicio	Responsabilidad	Persistencia
service-a	Coordina el pedido, expone la sesión y reenvía la cabecera de experimento.	Ninguna.
service-b	Prepara el cumplimiento. Único lugar de la espera de 200 ms.	Ninguna.
data-service	Lee el pedido y el inventario. Único lugar del error inyectado y del JDBC.	MySQL, una URL por perfil.

Tabla 2. Responsabilidad de cada proceso.

3.1 Perfiles de base de datos

El perfil local es el de Compose y el del MySQL que el manifiesto de GKE despliega dentro del clúster. La URL sale de SPRING_DATASOURCE_URL. El perfil gcp no tiene valor por defecto: exige CLOUD_SQL_JDBC_URL y solo tiene sentido si create_cloud_sql se hubiera aplicado. El perfil aws exige RDS_JDBC_URL. Si create_rds permanece en false, esa URL apunta al MySQL que viaja en la misma tarea ECS. Ese sidecar no es un RDS simulado desde el portátil, y el MySQL del compose no es Cloud SQL.

Perfil	Variable de URL	Cuándo se usa
local	SPRING_DATASOURCE_URL	Compose y MySQL del manifiesto GKE.
gcp	CLOUD_SQL_JDBC_URL	Solo con Cloud SQL creado. Sin fallback local.
aws	RDS_JDBC_URL	RDS si create_rds es true; sidecar de la tarea si no.

Tabla 3. Un proceso no combina los perfiles gcp y aws.

3.2 Pipeline de señales

Cada proceso exporta OTLP gRPC a otel-collector:4317. El muestreo de cabeza es parentbased_traceidratio con argumento 1.0, para conservar todas las trazas del laboratorio. El Collector limita la memoria a 400 MiB, inserta el entorno local y el namespace lab33, descarta los spans de salud y de /api/chaos, agrupa y exporta. Las trazas van por otlp_grpc/tempo. Los logs pasan por un transform que escribe trace_id= en el cuerpo y salen por otlp_http/loki. Las métricas se exponen en el puerto 8889 con etiquetas constantes de service.name, versión, entorno, namespace y cloud.provider. Prometheus las scrapea. Grafana lee Prometheus, Tempo y Loki.

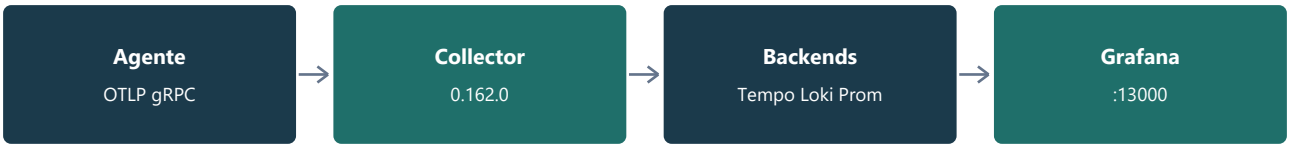


Figura 2. Camino de las tres señales. El Collector no instrumenta la aplicación: la recibe y la reparte.

Componente	Versión en el repositorio
Java y Spring Boot	21 y 3.5.16
API OpenTelemetry en el jar	1.65.0. El SDK de runtime lo aporta el agente, no una segunda dependencia.
Java Agent	2.31.1
Collector contrib	0.162.0
MySQL, Tempo, Loki	8.4, 2.8.1 y 3.4.2
Prometheus y Grafana	v3.4.1 y 11.6.0

Tabla 4. Versiones fijadas en el pom, el Dockerfile y el compose local.

4. Instrumentación OpenTelemetry

El jar depende del API. Incluir además el SDK y el agente duplicaría providers. entrypoint.sh añade -javaagent solo si OTEL_ENABLED no es false. Así el mismo artefacto puede arrancar sin telemetría, aunque este laboratorio no usa esa vía como medición de overhead: no se ejecutó un benchmark.

4.1 Qué abre el agente y qué abre el código

El agente abre el span servidor de Tomcat y Spring MVC, el span cliente de la llamada saliente y el span de base de datos sobre el driver JDBC. Esos spans de base siguen las convenciones que trae el agente 2.31.1: sistema, nombre de operación y sentencia, sin que el código copie el SQL en un span manual. Los spans escritos a mano son order.coordinate en service-a, fulfillment.prepare en service-b y record.load en data-service. Nombran la decisión de negocio. No se usa @WithSpan encima de esos mismos bloques.

Las rutas de actuator y de /api/chaos están en OTEL_JAVAAGENT_EXCLUDE_URLS. El filtro del Collector repite el descarte de salud y de caos. El motivo es que el control del experimento no debe entrar en el SLI de /api/orders/ ni en el error rate de /api/records/.

4.2 Métricas, logs y recursos

El agente exporta el histograma http.server.request.duration, métricas JVM y los contadores que el código registra: fallos de autenticación, inyecciones de latencia e inyecciones de error. El intervalo de exportación es 15 segundos. No hay un segundo scrape de Micrometer en /actuator/prometheus: el actuator solo expone salud. Prometheus lee al Collector.

Logback escribe JSON con nivel, nombre de servicio, entorno y, cuando el contexto tiene un span válido, trace_id y span_id. El recurso del proceso fija service.namespace=lab33, service.version=1.0.0, deployment.environment=local y cloud.provider=local. En los manifiestos cloud esas etiquetas cambian con el proveedor; no se afirmó haberlas visto en un clúster real.

4.3 Propagación

OTEL_PROPAGATORS vale tracecontext,baggage. El agente escribe la cabecera traceparent en el cliente HTTP. El servicio siguiente extrae el mismo trace_id. service-a reenvía además la cabecera de experimento X-Lab-Experiment cuando la petición de entrada la trae. Esa cabecera no es contexto de traza: es la marca que limita el error del 10 % a solicitudes identificables.

5. Correlación entre métricas, logs y trazas

La correlación exige el mismo identificador de 32 hexadecimales en la respuesta, en el log y en Tempo, y un enlace de Grafana que sepa leerlo. Imprimir un UUID que no viaja en la cabecera W3C no correlaciona nada.

5.1 Cómo nace el identificador

El agente crea el trace_id al recibir la petición en service-a. order.coordinate hereda ese contexto porque se abre en el hilo activo. RestClient no compone la cabecera a mano. service-b y data-service continúan la traza. La respuesta JSON incluye traceId para copiarlo sin abrir los logs. Dos span_id distintos con el mismo trace_id son el signo de que la cadena avanzó.

5.2 Del log a la traza y de la traza al log

El processor transform/logs copia el identificador al atributo y al cuerpo con el prefijo trace_id=. El datasource de Loki tiene un derived field cuya expresión es trace_id=([0-9a-f]{32}) y cuyo destino es Tempo. En el YAML de aprovisionamiento el dólar se escribe como $$. Grafana sustituye ${...} por variables de entorno al cargar el archivo; un solo dólar vacía la URL y el salto deja de existir. Tempo declara tracesToLogsV2 hacia Loki, filtrando por el trace id de la traza abierta, con una ventana de dos minutos. Jaeger no forma parte del compose.

5.3 De la métrica a la traza

El exporter Prometheus del Collector tiene OpenMetrics activado y Prometheus arranca con exemplar-storage. El datasource declara el destino del exemplar hacia Tempo. Eso prepara el salto desde un punto del gráfico. Hasta ver el exemplar en Explore no se afirma que el agente lo esté adjuntando. El camino que el código sí garantiza, cuando el stack está arriba, es copiar el traceId de la respuesta y buscarlo en Loki y en Tempo.

Cómo se investiga un incidente con este diseño. Se parte de la alerta y de su etiqueta anomaly_type. Si la alerta es de latencia, se mira el p99 de /api/orders/ y el span fulfillment.prepare, que lleva chaos.delay_ms cuando la espera fue inyectada. Si la alerta es de error, se busca en Loki la línea error inyectado de data-service. El script observability/aiops/enrich_incident.py lee las alertas de Prometheus y, si Loki tiene una línea reciente, adjunta el trace_id. Si no hay línea, el incidente sigue siendo válido y dice expresamente que no encontró traza. Prometheus no guarda ese identificador en la serie: meterlo en la regla PromQL no es posible con estas métricas.

Esta secuencia está implementada y no está demostrada con una petición real de esta sesión. La comprobación pendiente es un GET de ord-001, el mismo traceId en los logs de los tres servicios, en Loki y en Tempo, con el span JDBC debajo de record.load.

6. AIOps y detección de anomalías

Cloud Monitoring puede detectar anomalías sobre series que ya estén en ese servicio, y Amazon DevOps Guru analiza recursos de AWS que le hayan sido habilitados. Ninguno de los dos está disponible en este entorno: no hay proyecto con facturación ni un apply. La detección reproducible es la de Prometheus, y las reglas quedan como el contrato que un adaptador cloud tendría que respetar.

6.1 Definiciones de la regla

Término	Definición en este laboratorio
error_rate	Cociente de respuestas 5xx y peticiones totales de data-service en la ruta /api/records/, calculado con rate en 2 minutos. Si no hay 5xx, el numerador es 0.
baseline	avg_over_time de ese cociente durante 10 minutos.

Término	Definición en este laboratorio
σ	stddev_over_time del mismo cociente durante 10 minutos.
p99	histogram_quantile(0.99) del histograma de service-a en /api/orders/, con rate de 2 minutos.
Umbral de latencia	150 ms. Umbral de diseño: la inyección de 200 ms debe cruzarlo. No es un p99 medido en este laboratorio.
SLO de éxito	99 % de respuestas 2xx en /api/orders/.
Historia mínima	count_over_time de la serie de error en 10 minutos mayor o igual que 8, y desviación igual a sí misma para descartar NaN.

Tabla 5. Términos de la regla correlacionada. Las ventanas no son de unos segundos: diez minutos de baseline evitan una desviación inestable.

La alerta DataServiceCorrelatedAnomaly se dispara solo si se cumplen a la vez el exceso sobre baseline + 2σ, el p99 de service-a por encima de 150 ms, las ocho muestras y la desviación definida. El valor que PromQL conserva en un and es el del lado izquierdo, así que la anotación error_rate muestra el cociente de error, no la latencia. La latencia va en la anotación slo_seconds, fija en 0,150.

La alerta lleva servicio, entorno, plataforma, tipo de anomalía, severidad critical y una recomendación: revisar si el caos sigue activo, luego la traza del error inyectado y el SELECT. El identificador de incidente lo construye el script de enriquecimiento al leer la alerta; la regla de Prometheus no inventa un UUID. El enlace operativo es la consulta de Loki y la de Tempo anotadas en la propia alerta. Cuando el script no encuentra traza, lo dice y no fabrica un trace_id.

6.2 Umbral fijo frente a correlación

Conviven tres alertas de confiabilidad. ServiceALatencySlo exige p99 mayor que 150 ms durante 30 segundos. DataServiceErrorStatic exige error rate mayor que 5 % durante 30 segundos. La correlacionada exige las dos familias de síntoma y, además, historia. El diseño permite un experimento de solo latencia, uno de solo error y uno combinado, sobre el mismo tráfico y las mismas ventanas. La hipótesis es que la latencia sola encienda la alerta estática de SLO y no la correlacionada, porque el error rate seguiría cerca de cero; que los errores solos enciendan el umbral del 5 % sin cruzar 150 ms si data-service responde rápido; y que solo la combinación cumpla la regla estadística.

Esa comparación no se ejecutó. No hay un archivo de resultados con conteos. Este informe no declara una reducción porcentual de ruido. El procedimiento para obtenerla está en chaos/run_scenarios.py: tres pasadas, tope de 150 segundos cada una, y un JSON que solo se escribe si el stack responde. Hasta que ese JSON exista, la comparación es un diseño, no un resultado.

La alerta de autenticación es independiente: increase(auth_failures_total[5m]) mayor que 3. El contador lo incrementa el rechazo del token. No se alimenta con un log inventado.

7. Observabilidad de red y de seguridad

Hay cuatro capas y no deben mezclarse. La telemetría de aplicación es la del agente. La telemetría L7 de un proxy saldría de Cloud Service Mesh o de ECS Service Connect, y ninguna de las dos está aplicada. La telemetría de infraestructura de red, en cloud, serían los VPC Flow Logs, escritos en Terraform y no creados. Los eventos de seguridad de plataforma serían Security Command Center y Security Hub, que exigen activación, permisos y, en el caso de Hub, servicios que generen hallazgos. No están activos y este informe no les atribuye hallazgos.

7.1 Por qué no hay mesh desplegado

AWS App Mesh dejó de aceptar clientes nuevos el 24 de septiembre de 2024 y el soporte terminó el 30 de septiembre de 2026. No forma parte de la solución. La alternativa vigente en ECS es Service Connect. No está en la definición del servicio porque la tarea de laboratorio coloca los tres procesos en la misma red awsvpc y se hablan por 127.0.0.1. Ese tráfico no atraviesa un proxy. Separar cada proceso en su propio servicio ECS tiene costo de tareas y queda como paso posterior.

En GKE, Cloud Service Mesh es el mesh gestionado basado en Istio. No está en el kustomization: inyectar sidecars sin el fleet deja los pods sin arrancar. Hasta entonces, el tráfico este-oeste local se observa con http_client_request_duration del agente.

7.2 Qué muestra el tablero y de dónde sale

Panel	Fuente real	Qué no representa
Autenticaciones rechazadas	Contador auth.failures y respuestas 401 de POST /api/session.	No es Cloud Audit ni CloudTrail.
Tráfico norte-sur	Histograma HTTP servidor de service-a.	No son flow logs de la VPC.
Tráfico este-oeste	Histograma HTTP cliente de service-a y service-b.	No son métricas de Envoy.
Errores 401, 403 y 5xx de cliente	Códigos del histograma del agente.	No son hallazgos de Security Hub.
CVE	Panel de texto: no hay fuente.	No hay un entero inventado.

Tabla 6. Cada panel del tablero de seguridad declara su fuente. Sin stack en marcha, los paneles no tienen serie observada.

El rechazo de sesión no escribe el token. La comparación usa una igualdad en tiempo constante y el log dice autenticacion rechazada. Las contraseñas de MySQL viven en variables de entorno del compose y en un ejemplo de secreto de Kubernetes que no debe aplicarse tal cual. No se registran en los logs de aplicación.

En GCP, el Terraform de la subred incluye flow logs con muestreo 0,5. Cloud SQL está detrás de create_cloud_sql, en false, con IP privada y protección contra borrado. Activarlo sin el emparejamiento de Service Networking fallaría en el apply; por eso el informe no lo da por utilizable. En AWS, flow_logs.tf envía los flujos de la VPC a un log group propio, con un rol que solo puede asumir el servicio de flow logs. CloudTrail y Security Hub quedan como requisitos de una cuenta de laboratorio, no como recursos ya creados. Crearlos tiene costo y no se hizo.

8. Chaos engineering, MTTD, SLO y error budget

Los dos experimentos nacen apagados. CHAOS_ARMED es false en el compose base. Aunque alguien llame al POST de latencia o de errores, la respuesta es 409. El override docker-compose.chaos.yml arma el proceso y hay que pedirlo de forma explícita. Armar no inyecta: hace falta un segundo POST con la cabecera X-Lab-Token y X-Lab-Enable en true. El token se compara y no se escribe en el log. POST /api/chaos/stop apaga las dos inyecciones sin reconstruir la imagen.

8.1 Experimento de latencia

La espera es de 200 ms, constante, dentro de fulfillment.prepare. El contador chaos.latency.injections y el campo latencyInjectedMs de la respuesta distinguen esa espera de la latencia normal. service-a se alarga porque espera a service-b. data-service no duerme. El efecto previsto sobre el SLO es cruzar los 150 ms de p99 y poder encender ServiceALatencySlo. La traza debería mostrar chaos.delay_ms=200. Nada de eso se observó en contenedores: la prueba unitaria de service-b sí comprobó que, con el proceso armado, la llamada espera al menos 180 ms y registra 200, y que desarmado no espera.

8.2 Experimento de error del 10 %

La probabilidad es la constante 0,10. Solo aplica si el proceso está armado, el interruptor de errores está encendido y la petición trae X-Lab-Experiment: error-rate. Una petición sin esa cabecera no entra en el sorteo. El fallo ocurre antes del repositorio, así que no es un error de SQL disfrazado. La prueba unitaria fuerza el aleatorio: con un valor menor que 0,10 el servicio falla sin llamar a la base; con un valor mayor o igual, no falla. En una muestra corta de veinte peticiones el porcentaje observado puede alejarse de 10. La cifra que se defiende es la probabilidad configurada, no el recuento de una muestra pequeña.

8.3 Qué se considera detección

El script chaos/run_scenarios.py toma started_at al iniciar el escenario. La primera degradación observable es una respuesta HTTP 500 o una duración de al menos 180 ms. La detección es el instante en que Prometheus

muestra la alerta esperada en estado firing. mttd_seconds es la diferencia. Si la alerta no aparece en 150 segundos, el campo queda null. No se sustituye por 120. El objetivo de diseño es un MTTD de dos minutos o menos para la alerta estática, cuyo for es de 30 segundos y cuya ventana de rate es de 2 minutos. La alerta correlacionada necesita del orden de diez minutos de historia; no es el instrumento con el que se pretende cumplir los dos minutos. En esta sesión el script no se ejecutó. El MTTD está pendiente.

8.4 SLO y presupuesto, con la única cifra calculada

El SLI de éxito es la fracción de respuestas 2xx de /api/orders/ sobre todas las respuestas de esa ruta. El SLO es 99 %. El presupuesto de una ventana es el 1 % de sus peticiones. La fracción consumida es fallos divididos entre ese presupuesto. La extrapolación multiplica el consumo del experimento por la duración del experimento y lo divide entre la ventana del SLO. No son la misma cifra.

ErrorBudgetTest fija 1000 peticiones, 10 fallos y SLO 0,99. El presupuesto es 10 peticiones y la fracción consumida es 1,0: esa ventana hipotética agota el presupuesto. La extrapolación de 120 segundos a 30 días es 120 / 2 592 000. Esa prueba demuestra la aritmética. No demuestra que un experimento de este laboratorio haya producido 10 fallos, ni que el SLO se haya degradado en un entorno en marcha. Cuando exista evidence/chaos/resultados.json, el consumo deberá calcularse con los totales de esa corrida y etiquetarse como ventana experimental, distinto de la extrapolación a 30 días.

Magnitud	Valor	Naturaleza
SLO de éxito	99 % de 2xx	Definición.
Umbral de latencia	150 ms	Diseño. No medido en ejecución.
Latencia inyectada	200 ms	Constante de código. Comprobada en unidad.
Probabilidad de error	0,10, solo con cabecera	Constante de código. Comprobada en unidad.
Consumo en el test	1,0 para 10 fallos de 1000	Prueba de la fórmula. No es una corrida.
MTTD	Pendiente	El script no se ejecutó. No se rellena.
Reducción de ruido	Pendiente	No hay conteo de alertas firing.

Tabla 7. Separación entre definiciones, pruebas unitarias y mediciones que faltan.

La recuperación prevista es el POST de parada y, si hiciera falta, volver al compose sin el override para dejar CHAOS_ARMED en false. Después, un pedido ord-001 debe responder 200 con latencyInjectedMs en 0. Esa comprobación también está pendiente de un stack en ejecución. Los experimentos no deben lanzarse contra un servicio productivo ni contra una base cloud sin una cuenta de laboratorio y un alcance revisado. En esta sesión no se lanzaron.

9. Evaluación de madurez

No hay en este repositorio, ni apareció en la búsqueda pública hecha para este trabajo, un documento oficial llamado Observability Foundation Blueprint con una lista cerrada de ocho dominios. La escala aplicada es la de la actividad, de 1 a 5: 1 inicial, 2 básico y reactivo, 3 estandarizado y repetible, 4 avanzado y correlacionado, 5 optimizado y automatizado. Los ocho dominios se definen aquí. No son una clasificación de un fabricante. La nota no sube por la existencia de un YAML.

Dominio	Nivel	Evidencia y techo
Telemetría de aplicación	3	Agente, spans y logs están en el código y las pruebas pasan. El pipeline no se observó: el registro de imágenes respondió 504.
Correlación	3	Derived fields, tracesToLogsV2 y el script de enriquecimiento están escritos. Falta el salto en Grafana con un trace_id real.
Detección	2	Las reglas existen. No hay una alerta firing.
SRE	2	SLO, presupuesto y fórmula están probados en unidad. No hay ventana experimental.
Seguridad y red	2	El 401 y el contador son código real. Flow logs y centros de seguridad cloud no están desplegados. No hay CVE.

Dominio	Nivel	Evidencia y techo
AIOps	2	La regla estadística está versionada. No hubo conteo de alertas estáticas frente a correlacionadas.
Caos y recuperación	2	La política está probada. El MTTD contra Prometheus no se midió.
Plataforma multicloud	2	Terraform, manifiestos y perfiles están en el repositorio. No hubo apply.

Tabla 8. Ningún dominio está en 4 o 5. La correlación diseñada no sustituye a una traza observada.

El riesgo operativo de dejar la nota donde está es menor que el de declararla alta. Un nivel 4 exigiría haber seguido un incidente real de este sistema, con alerta, traza y recuperación medidas. Hoy el riesgo concreto es otro: creer que el compose está sano porque los archivos existen, y descubrir el 504 del registro el día de la demostración. El segundo riesgo es aplicar Cloud SQL o RDS sin revisar el costo y el emparejamiento de red.

10. Roadmap de 90 días

El orden prioriza detección, diagnóstico y recuperación. El mesh y los flow logs vienen después, porque no desbloquean la primera evidencia. Cada quincena cabe en el repositorio actual; no exige reescribir los servicios.

Quincena	Acción	Prioridad	Éxito
1	Levantar el compose cuando el registro entregue las imágenes Java y guardar una traza con los tres servicios y el span JDBC.	Alta	El mismo trace_id en la respuesta, en Loki y en Tempo.
2	Ejecutar chaos/run_scenarios.py y archivar el JSON.	Alta	mttd_seconds numérico, o null explícito si la alerta no apareció. Sin rellenar 120.
3	Separar los tres procesos ECS y activar Service Connect en un namespace de laboratorio.	Media	Métricas L7 del proxy distintas de las del agente, en una cuenta autorizada.
4	Activar Cloud Service Mesh solo con el fleet ya creado y comparar sus métricas con las del agente.	Media	Pods en Ready y una serie L7 que no sea http_client del agente.
5	Encender VPC Flow Logs en la cuenta de laboratorio y un panel de rechazos con esa fuente.	Media	Registros de flujo reales, no una consulta vacía presentada como hallazgo.
6	Revisar el umbral de 150 ms con el p99 medido en la quincena 1 y alargar el baseline si la desviación oscila.	Alta	El umbral queda justificado con un percentil de este laboratorio.

Tabla 9. Seis quincenas. El éxito de la segunda es un archivo, no una estimación.

Dependencias: la quincena 1 depende de Docker Hub o de imágenes ya cargadas en la máquina. La 2 depende de la 1 y de varios minutos de tráfico sano, porque la alerta correlacionada pide historia. La 3 y la 4 dependen de permisos y de un tope de gasto. La 5 depende de la red creada por Terraform. La 6 depende de la traza y de las series de la quincena 1. El esfuerzo de las dos primeras cabe en una sesión de laboratorio si el registro responde. El de las siguientes es de días, no de horas, porque incluye revisión de IAM y de costo.

11. Conclusiones y recomendaciones

El laboratorio deja un sistema coherente: una sola base por proceso, tres spans de negocio que no repiten el SQL, un caos con dos compuertas y una regla de anomalía que no puede dispararse con una serie recién nacida. Las pruebas unitarias sostienen esas compuertas y la aritmética del presupuesto. Eso es suficiente para defender el diseño. No es suficiente para defender un tiempo de detección, una captura o un ahorro de alertas.

La decisión de descartar App Mesh está fechada en el fin de soporte del 30 de septiembre de 2026. Sustituirlo por un mesh no aplicado habría sido peor que declarar que, hoy, la observabilidad L7 disponible en el diseño local es la del agente. La misma franqueza vale para los CVE y para los centros de seguridad: sin fuente, no hay panel numérico.

Recomendaciones, en el orden en que desbloquean la sustentación:

- Repetir docker compose cuando el registro responda. Comprobar ord-001, el traceId compartido y el span JDBC antes de grabar.
- Armar el caos solo con el override, ejecutar los dos experimentos, pararlos y dejar un pedido sano. No grabar el tramo de contenedores si el 504 se repite.
- Ejecutar chaos/run_scenarios.py y conservar el JSON aunque mttd_seconds sea null.
- No calcular un porcentaje de ruido hasta tener los tres escenarios en ese archivo.
- No ejecutar terraform apply en el ensayo. Si más adelante hay cuenta, empezar por plan, con create_cloud_sql y create_rds en false.
- Revisar el umbral de 150 ms en cuanto exista un p99 de este stack. Hasta entonces, presentarlo como umbral de diseño.

El aprendizaje principal es de método. Una alerta enriquecida se diseña con las señales que de verdad existen: el trace_id vive en el log, no en el histograma. Un experimento seguro se diseña para que el estado por defecto sea no hacer daño. Y una nota de madurez se escribe después de la corrida, no antes.

12. Referencias técnicas

Las versiones citadas son las fijadas en este repositorio. Las fechas de App Mesh son las publicadas por AWS y se usan para justificar una decisión de diseño, no como resultado de un despliegue.

- OpenTelemetry Java Instrumentation, agente 2.31.1, y API 1.65.0. Documentación de configuración del agente: service name, resource attributes, propagators y muestreo parentbased_traceidratio.
- OpenTelemetry Collector Contrib 0.162.0. Exporters otlp_grpc y otlp_http, exporter Prometheus con resource_constant_labels y translation_strategy UnderscoreEscapingWithSuffixes.
- Convenciones semánticas de OpenTelemetry para HTTP y para bases de datos, en la generación que instrumenta el agente 2.31.1. El código no redefine el span JDBC.
- Spring Boot 3.5.16 y Java 21. El actuator se limita a salud para no duplicar el histograma HTTP.
- Prometheus: recording rules, histogram_quantile, avg_over_time, stddev_over_time y el operador and.
- Grafana 11.6: aprovisionamiento de datasources, derived fields y el escape $$ de las variables de consulta.
- Grafana Tempo 2.8.1 y Grafana Loki 3.4.2, incluido el receptor OTLP de Loki.
- MySQL 8.4, imagen oficial usada por el compose local.
- AWS, documentación de fin de soporte de App Mesh (30 de septiembre de 2026) y de Amazon ECS Service Connect.
- Google Cloud, documentación de Cloud Service Mesh y de Cloud SQL para MySQL. El recurso de Terraform permanece desactivado.
- Google Cloud y AWS, documentación de VPC Flow Logs. Los recursos están en los archivos de Terraform y no se aplicaron.

Archivos de este repositorio que sostienen el informe: deploy/local/docker-compose.yaml, observability/collector/collector-local.yaml, observability/prometheus/rules/lab33-alerts.yml, observability/aiops/enrich_incident.py, chaos/run_scenarios.py, docs/arquitectura.md, docs/madurez.md y PASOS.md. El generador de este PDF es docs/generar_reporte_pdf.py.